

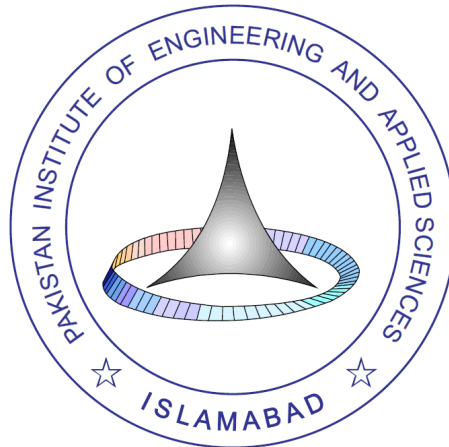
LLM-AUGMENTED MALWARE ANALYSIS SYSTEM

By

FATIMA KAISER (03-3-1-016-2022)

BISMA TAHIR (03-3-1-011-2022)

THESIS SUBMITTED TO THE FACULTY OF ENGINEERING AT PIEAS IN PARTIAL
FULFILLMENT OF REQUIREMENTS FOR THE DEGREE OF BS COMPUTER AND
INFORMATION SCIENCES .



DEPARTMENT OF COMPUTER AND INFORMATION SCIENCES ,
PAKISTAN INSTITUTE OF ENGINEERING AND APPLIED SCIENCES,
NILORE, ISLAMABAD 45650, PAKISTAN

JUNE 2026

Department of Computer and Information Sciences

Pakistan Institute of Engineering and Applied Sciences (PIEAS)

Nilore, Islamabad 45650, Pakistan

Declaration of Originality

We hereby declare that the work contained in this thesis and the intellectual content therein are the product of our own research. This thesis has not been previously published in any form, nor does it contain any verbatim of the published resources that could be treated as an infringement of international copyright law. We also declare that we do understand the terms '*copyright*', '*plagiarism*', and that, in case of any copyright violation, plagiarism, or significant AI-generated content in this work, we will be held fully responsible for the consequences of any such violation.

Name: Fatima Qaiser

Signature: _____

Name: Bisma Tahir

Signature: _____

Date: June 5, 2026

Place: PIEAS

Certificate of Approval

This is to certify that the work contained in this thesis entitled

“LLM-Augmented Malware Analysis System ”

carried out by

Fatima Qaiser (03-3-1-016-2022)

Bisma Tahir (03-3-1-011-2022)

*It is fully adequate, in scope and quality, for the degree of BS Computer
and Information Sciences from Pakistan Institute of Engineering and
Applied Sciences (PIEAS).*

Approved By:

Signature: _____

Academic Supervisor: Dr. Muhammad Abid Mughal

Signature: _____

Co-Supervisor: Dr. Nouman Shamim

Verified By:

Signature: _____

Head, Department of Computer and Information Sciences

Stamp:

To parents, teachers, and muslims of Palestine

Acknowledgements

We begin by expressing our deepest gratitude to Allah Almighty for His mercy and blessings that enabled us to complete this research.

We sincerely thank our supervisor, **Dr. Muhammad Abid Mughal**, for his invaluable mentorship, guidance, and constant encouragement throughout this project. We are equally grateful to our co-supervisor, **Dr. Nouman Shamim**, for his valuable suggestions and technical insights. We also thank the faculty and administrative staff of our department for their support and cooperation.

From Fatima Qaiser: I am deeply grateful to my parents and my phapphy for their endless prayers, love, and unwavering support throughout this journey.

From Bisma Tahir: I would like to express my heartfelt gratitude to my parents for their endless prayers, patience, and belief in me. Their encouragement has been a constant source of strength throughout this journey.

Finally, we acknowledge everyone who contributed, directly or indirectly, to the completion of this thesis.

Fatima Qaiser

Bisma Tahir

Contents

List of Figures	x
List of Tables	xi
1 Introduction	1
1.1 Background and Motivation	1
1.2 Goals and Objectives	2
2 Literature Review	4
2.1 Traditional Malware Analysis	4
2.2 Machine-Learning-Based Static Analysis of PE Files	5
2.3 Packing and Obfuscation Detection	6
2.4 Network Traffic Classification for Malware Detection	7
2.5 LLM-Augmented and Multi-Agent Cybersecurity Systems	8
3 Methodology	11
3.1 System Overview	11
3.2 Agent Descriptions	13
3.2.1 Reputation Agent	13
3.2.2 Semantic Analysis Agent	13
3.2.3 Packing Detect Agent	13
3.2.4 Extraction Agent	13
3.2.5 Network Behaviour Agent	14
3.3 Reputation Agent	14
3.3.1 Purpose and Role	14

3.3.2	VirusTotal Integration	14
3.3.3	LLM-Based Analysis	15
3.3.4	Output	15
3.3.5	Implementation Summary	16
3.4	Network Behaviour Agent	16
3.4.1	Purpose and Role	16
3.4.2	Dataset	17
3.4.3	Feature Categories	17
3.4.4	Dataset Preparation and Sampling	19
3.4.5	Natural Language Conversion	19
3.4.6	Supervised Fine-Tuning	19
3.4.7	Inference	21
3.4.8	Implementation Summary	22
3.5	Packing Detect Agent	22
3.5.1	Purpose and Role	22
3.5.2	Detection Strategy	23
3.5.2.1	Shannon Entropy Analysis	23
3.5.2.2	YARA Signature Matching	24
3.5.2.3	Structural Heuristics	25
3.5.3	Scoring and Verdict Derivation	26
3.5.4	Output	26
3.5.5	Implementation Summary	27
3.6	Packing Detect Agent: Case Study	28
3.6.1	Test Input	28
3.6.2	Agent Output	29
3.6.2.1	Entropy Analysis	29
3.6.2.2	YARA Signature Matching	29
3.6.2.3	Structural Heuristics	29
3.6.2.4	Verdict and Score	30

3.6.3	Discussion	30
3.7	String Extraction Agent	31
3.7.1	Purpose and Role	31
3.7.2	String Extraction	32
3.7.3	IOC Identification	32
3.7.4	LLM-Based Behavioural Classification	33
3.7.5	IOC Threat Scoring	35
3.7.6	Output	35
3.7.7	Implementation Summary	36
4	Results	37
4.1	Reputation Agent: Case Study	37
4.1.1	Test Input	37
4.1.2	VirusTotal Query Result	38
4.1.3	LLM-Generated Analysis	38
4.1.4	Discussion	39
4.2	Network Behaviour Agent: Case Study	40
4.2.1	Test Input	40
4.2.2	Model Output	41
4.2.3	Discussion	41
4.3	Packing Detect Agent: Case Study	42
4.3.1	Test Input	42
4.3.2	Agent Output	42
4.3.2.1	Entropy Analysis	42
4.3.2.2	YARA Signature Matching	43
4.3.2.3	Structural Heuristics	43
4.3.2.4	Verdict and Score	44
4.3.3	Discussion	44
4.4	String Extraction Agent: Case Study	45
4.4.1	Test Input	45

4.4.2	Agent Output	45
4.4.2.1	String Extraction	45
4.4.2.2	IOC Identification	46
4.4.2.3	LLM Behavioural Classification	47
4.4.2.4	IOC Threat Score	47
4.4.3	Discussion	48
5	Conclusion	50
	References	51
	Appendices	54
	About the Authors	55

List of Figures

Figure 3.1	High-level architecture of the LLM-MAS system.	12
Figure 3.2	Class distribution of malware families in the BCCC network traffic dataset.	17
Figure 4.1	Output of the Reputation Agent for the EICAR standard antivirus test file hash.	38
Figure 4.2	Output of the Network Behaviour Agent for a malicious traffic flow sample.	41

List of Tables

Table 3.1	Representative samples from the supervised fine-tuning dataset. . .	20
Table 3.2	Training configuration for supervised fine-tuning of Mistral 7B. . .	21
Table 3.3	Training and validation loss across three epochs.	21
Table 3.4	Per-section Shannon entropy values for the UPX-compressed test binary.	29
Table 4.1	Per-section Shannon entropy values for the UPX-compressed test binary.	43
Table 4.2	IOCs extracted from the Mirai test binary by the String Extraction Agent.	46

Abstract

Modern malware increasingly employs obfuscation, polymorphism, and anti-analysis mechanisms that render traditional signature-based detection and manual reverse engineering insufficient for timely threat response. This thesis proposes an LLM-Augmented Multi-Agent Malware Analysis System (LLM-MAS). The LLM-MAS is a modular and intelligent architecture designed for automated malware analysis and threat detection. The system involves five specialized components: the Reputation Agent checks file hashes using VirusTotal and LLM-based analysis for assessment, the Semantic Analysis Agent performs high-level reasoning on PE features to understand intent, and the Packing Detect Agent identifies obfuscation using entropy analysis and YARA rules. Additionally, the Extraction Agent pulls artifacts like strings and URLs from the code, while the Network Behavior Agent uses a fine-tuned LLM to classify traffic as benign or malicious. Finally, outputs from all agents are merged to generate a comprehensive, accurate, and explainable final report.

Chapter 1

Introduction

1.1 Background and Motivation

The proliferation of malicious software has become one of the most pressing challenges in modern cybersecurity. Malware, defined broadly as any software designed to disrupt, damage, or gain unauthorised access to a computer system, has evolved considerably in both sophistication and scale over the past decade. Traditional detection approaches, principally signature-based matching and manual reverse engineering, were developed in an era when malware variants were relatively few and behavioural patterns were comparatively stable. These methods, however, are increasingly inadequate in the face of contemporary threats that are designed to evade static detection through obfuscation, packing, polymorphism, and metamorphism.

Polymorphic malware continuously mutates its code structure while preserving its underlying functionality, thereby rendering signature databases obsolete against each new variant. Metamorphic malware goes further by rewriting its own logic entirely between executions. Beyond evasion at the code level, modern malware employs packing techniques that conceal executable logic within compressed or encrypted containers, significantly slowing manual analysis. The emergence of zero-day exploits, which exploit previously unknown vulnerabilities and therefore have no existing signatures, compounds the problem further. Analysts are simultaneously confronted with thousands of new mal-

ware samples daily, a volume that is impossible to address through manual investigation alone.

The limitations of existing approaches have created an urgent need for automated, intelligent, and scalable malware analysis systems. Large language models (LLMs) have recently demonstrated remarkable capabilities in natural language understanding, code interpretation, and contextual reasoning, making them well suited for augmenting static and dynamic analysis pipelines. When combined with a multi-agent architecture in which specialised agents handle distinct subtasks, such a system can provide both breadth and depth of analysis that is infeasible through conventional means. The present project is motivated by this gap between the capabilities of existing tools and the demands of the modern threat landscape.

1.2 Goals and Objectives

The primary goal of this project is to design, implement, and evaluate an LLM-augmented multi-agent system capable of automating malware analysis, accelerating threat detection, and delivering actionable intelligence to security practitioners.

The following are the specific objectives of this project:

1. To reduce reliance on manual reverse engineering by automating the malware analysis pipeline through a coordinated multi-agent framework in which each agent handles a distinct analysis task independently.
2. To leverage large language models for the interpretation of obfuscated code, detection of hidden behavioural patterns, and generation of structured, human-readable threat analysis reports that are accessible to security practitioners without requiring deep technical expertise.
3. To develop a scalable system architecture capable of handling large volumes of malware samples by distributing analysis responsibilities across specialised agents that operate concurrently on distinct input modalities.

4. To provide security teams with contextual and actionable intelligence by ensuring that all agent outputs are synthesised into a consolidated report that explains not only the classification decision but also the evidence and reasoning behind it.

Chapter 2

Literature Review

The research landscape surrounding automated malware analysis has evolved substantially over the past decade, transitioning from rule-based and signature-driven approaches toward machine-learning-based and, more recently, large language model (LLM)-augmented methodologies. This chapter surveys the body of prior work most relevant to the present system, organised across five thematic areas: traditional malware analysis techniques, machine-learning-based static analysis of Portable Executable (PE) files, packing and obfuscation detection, network traffic classification for malware detection, and LLM- and multi-agent-based cybersecurity systems.

2.1 Traditional Malware Analysis

Malware analysis has historically been divided into two complementary paradigms: static analysis, which examines binary artefacts without execution, and dynamic analysis, which observes program behaviour at runtime within a controlled environment. Static analysis techniques, including disassembly, string extraction, and PE header inspection, offer speed and safety but are fundamentally limited against obfuscated or packed executables [1]. Dynamic analysis, conducted within sandbox environments, provides richer behavioural evidence but incurs higher computational overhead and is susceptible to evasion by malware that detects the sandboxed environment.

Signature-based detection, the foundation of commercial antivirus software, maps known malicious byte sequences to curated databases. Although effective against catalogued threats, this approach fails categorically against polymorphic malware, which mutates its code between infections, and metamorphic malware, which entirely rewrites its logic [1]. The volume of new malware samples encountered daily has further rendered manual signature curation impractical. These fundamental limitations motivated the transition to data-driven detection methodologies that generalise beyond known signatures [2].

2.2 Machine-Learning-Based Static Analysis of PE Files

The application of machine learning to static PE-file analysis has produced a substantial body of literature. The PE header encodes rich structural information, including section entropy, import address table entries, data directory fields, and optional header attributes, that discriminates malicious from benign binaries without requiring execution [3]. Among tree-based ensemble methods, XGBoost has been consistently demonstrated to achieve state-of-the-art accuracy on PE-feature classification tasks. Yuk and Seo [3] proposed a stacking ensemble combining XGBoost with Extra Trees applied to PE header features, reporting classification accuracy exceeding 96%. Their investigation found that packer-derived features (pe_packer) contributed most significantly to discriminative power, motivating the integration of packing detection as an upstream pre-processing stage.

Ibrahim et al. [2] presented a comprehensive comparison of six machine learning classifiers operating over two feature categories extracted from PE headers and printable strings on the APT1 dataset. Their results confirmed that ensemble methods outperform single-classifier approaches, and that combined feature representations yield consistently higher detection rates. Similarly, Akinsowon and Jiang [4] employed BERT-based LLM text embeddings to represent dynamic malware behaviour reports, feeding the resulting vectors into Support Vector Machine, Multilayer Perceptron, and Random Forest classifiers. Their work demonstrated that LLM-derived embeddings capture semantic behavioural structure that conventional handcrafted features may miss.

A recurrent finding across this literature is the degradation of static PE-header features when applied to fine-grained intra-family discrimination. Experimental evidence indicates that while XGBoost achieves AUC scores above 0.99 for binary malware-versus-benign classification, its discriminative power diminishes when applied to structurally similar malware families [5]. This finding underscores the necessity of supplementing static feature analysis with semantic code understanding, as pursued in the present work through the Semantic Analysis Agent.

2.3 Packing and Obfuscation Detection

Packing constitutes one of the primary mechanisms by which malware evades static analysis. A packer compresses or encrypts the original executable payload, which is recovered only at runtime, concealing the malicious binary from static inspection tools. Entropy analysis is a widely employed heuristic for packer identification: sections with high Shannon entropy are likely to be compressed or encrypted [6]. Lyda and Hamrock established early empirical evidence for entropy-based packer detection, demonstrating that encrypted and packed sections reliably exhibit higher entropy than compiled code.

YARA, a pattern-matching framework designed for malware research, provides a complementary signature-based mechanism for packer identification. YARA rules describe structural and byte-sequence patterns characteristic of specific packer families, enabling rapid scanning without execution [7]. Naik et al. [7] proposed an enhancement of YARA rules using fuzzy hashing and fuzzy rule interpolation to improve robustness against minor variant mutations, demonstrating significant reductions in false negatives over conventional exact-match YARA rules.

Li et al. [6] introduced PackGenome, a framework that automatically generates robust YARA rules for packer detection from dynamic execution traces. Evaluated over 640,000 samples including Windows and Linux packed binaries, PackGenome achieved zero false negatives across all tested packer configurations, outperforming existing publicly available YARA rule collections. The framework demonstrates that combining entropy heuristics with automatically generated signature rules yields superior detection coverage compared

to either approach in isolation. The present system adopts this combined strategy within its Packing Detect Agent, applying Shannon entropy thresholds alongside YARA rule scanning to identify packed executables prior to semantic analysis.

2.4 Network Traffic Classification for Malware Detection

Network behaviour constitutes an independent and complementary observable surface for malware characterisation. Unlike static binary features, network traffic patterns are observable regardless of the degree of binary obfuscation, making traffic classification a robust detection modality for evasive samples. The foundational challenge in this domain is the increasing prevalence of encrypted traffic, which precludes deep packet inspection and necessitates flow-level statistical feature extraction [8].

Shen et al. [8] conducted a comprehensive survey of machine-learning-powered encrypted network traffic analysis, cataloguing flow-based feature extraction methods, classification architectures, and benchmark datasets. Their survey confirms that flow-based features, including packet inter-arrival times, byte counts per flow, and protocol flags, retain discriminative power even over encrypted channels. Machine learning classifiers trained on such features have been shown to distinguish malware traffic from benign traffic with high accuracy on standard benchmark datasets.

Wang et al. [9] demonstrated that convolutional neural networks trained on raw traffic representations could learn hierarchical features directly from byte sequences without domain-specific feature engineering. Subsequent work extended this approach to multi-modal feature fusion and graph-based traffic representations, with graph neural network approaches achieving strong performance on fine-grained traffic classification tasks [10]. The use of fine-tuned large language models for network traffic classification, as employed in the Network Behaviour Agent of the present system, is motivated by their ability to reason over natural language representations of flow features, capturing semantic patterns that conventional feature-based classifiers may overlook.

The Behaviour-Centric Cybersecurity Center (BCCC) dataset adopted in this work provides labelled network traffic flows spanning nine classes, comprising one benign cat-

egory and eight malware families: Backdoor, Exploit, HackTool, Hoax, Rootkit, Trojan, Virus, and Worm. The multi-class nature of this dataset presents a substantially more challenging classification problem than binary detection benchmarks, demanding models that distinguish subtle inter-family traffic differences in addition to identifying malicious traffic in general [11].

2.5 LLM-Augmented and Multi-Agent Cybersecurity Systems

The application of transformer-based language models and LLMs to cybersecurity tasks represents a rapidly maturing research frontier. Early work focused on adapting pre-trained code models to malware-adjacent tasks such as vulnerability detection and code summarisation. Subsequent investigations demonstrated that both general-purpose LLMs and domain-fine-tuned variants could be productively applied to malicious code analysis [1].

Patsakis et al. [12] systematically evaluated the capability of LLMs including ChatGPT and Code Llama for deobfuscation of real-world malware campaigns. Their findings established that LLMs can partially recover the semantic intent of obfuscated scripts, though they observed a tendency for models to generate deobfuscation code rather than directly interpret obfuscated content, reflecting limitations in reasoning over complex control-flow transformations. These limitations motivate the integration of specialised static analysis agents alongside the LLM reasoning component, as pursued in the present architecture.

Bitdefender Research [13] developed asmLLM, a domain-adapted language model based on MPT-7B fine-tuned on 500 million tokens of x86-64 assembly code using Low-Rank Adaptation (LoRA). Their results demonstrated that pretraining on assembly-specific data substantially improves downstream classification and code summarisation performance relative to general-purpose code LLMs, which tend to describe assembly line-by-line without capturing higher-level functional intent. This work establishes the importance of domain adaptation for assembly-level malware analysis.

Chen et al. [14] surveyed LLM applications across the cyber attack lifecycle, evaluating their utility for threat intelligence extraction, indicator-of-compromise classification, and attack attribution. Their taxonomy demonstrates that LLMs contribute most effectively to tasks requiring contextual natural-language reasoning, such as synthesising structured threat reports from heterogeneous evidence, which closely corresponds to the report generation role assigned to the LLM component in the present system.

At the system architecture level, several prior works have proposed multi-agent frameworks for cybersecurity. Tabaa et al. [15] presented a multi-agent system for cybersecurity threat detection and correlation published in IEEE Access, employing specialised agents for email verification, log analysis, and network anomaly detection, coordinated through a shared LLM-based reasoning backbone. Their system demonstrated that specialised task decomposition across agents with dedicated analysis capabilities yields superior detection accuracy compared to monolithic LLM-based detection approaches. The present system adopts the same architectural principle, decomposing the malware analysis workflow across five specialised agents covering distinct analysis modalities.

AgenticCyber [16] proposed a LangChain-orchestrated multi-agent framework integrating Gemini for multimodal inference across cloud logs, video, and audio streams. Their work confirms that LangChain provides an effective orchestration substrate for agent interaction, tool chaining, and retrieval-augmented generation in security-sensitive pipelines. ForenSift [17] similarly employed LangChain and LLMs to automate Digital Forensics and Incident Response (DFIR) workflows, demonstrating that LLM-driven evidence analysis and report generation substantially reduce investigation time compared to purely manual workflows.

The LLM-MalDetect framework proposed by [18], published in IEEE journals, addressed Android malware detection through structured prompt engineering and string-based feature extraction from APK files. Their approach formalises LLM adaptation via fine-tuning strategies that optimise semantic dependency modelling, achieving improved detection precision over prior LLM-based APK analysis methods.

Collectively, the existing literature establishes that no single analysis modality, whether static binary features, packing heuristics, network traffic patterns, or LLM-based code reasoning, achieves comprehensive malware detection in isolation. The present work synthesises these complementary modalities within a unified multi-agent architecture, addressing the integration gap that characterises prior single-modality or loosely coupled approaches.

Chapter 3

Methodology

This chapter describes the design and implementation of the LLM-Augmented Multi-Agent Malware Analysis System (LLM-MAS). The system is composed of five specialised agents, each responsible for a distinct analysis modality. The outputs of all agents are aggregated and synthesised into a consolidated threat assessment report. The following sections provide an overview of each agent, followed by detailed descriptions of the agents.

3.1 System Overview

The LLM-MAS architecture decomposes the malware analysis pipeline into five functionally independent but cooperating agents. Formally, the system is defined as a tuple:

$$\mathcal{S} = \langle \mathcal{A}, \mathcal{I}, \mathcal{O}, \mathcal{M}, \mathcal{R} \rangle \quad (3.1)$$

where $\mathcal{A} = \{a_1, a_2, \dots, a_5\}$ is the set of specialised agents, $\mathcal{I}$ is the set of input modalities, $\mathcal{O}$ is the set of agent output spaces, $\mathcal{M}$ is the merge function that aggregates agent outputs, and $\mathcal{R}$ is the report generation function implemented by the LLM component.

Each agent $a_i \in \mathcal{A}$ is modelled as a function:

$$f_{a_i} : \mathcal{I}_i \rightarrow \mathcal{O}_i \quad (3.2)$$

where $\mathcal{I}_i \subseteq \mathcal{I}$ is the input modality assigned to agent a_i and $\mathcal{O}_i \subseteq \mathcal{O}$ is its corresponding output space. The outputs of all agents are collected into an evidence vector:

$$\mathbf{e} = (f_{a_1}(\mathcal{I}_1), f_{a_2}(\mathcal{I}_2), \dots, f_{a_5}(\mathcal{I}_5)) \quad (3.3)$$

The merge function $\mathcal{M}$ aggregates the evidence vector into a unified context c :

$$c = \mathcal{M}(\mathbf{e}) \quad (3.4)$$

which is then passed to the report generation function $\mathcal{R}$ to produce the final threat intelligence report r :

$$r = \mathcal{R}(c) = \text{LLM}(\mathcal{M}(\mathbf{e})) \quad (3.5)$$

This formulation captures the core design principle of the system: the final report is derived not from any single analysis modality but from the aggregated evidence of all five agents, ensuring comprehensive coverage across complementary analysis perspectives. Figure 3.1 illustrates the high-level system architecture.

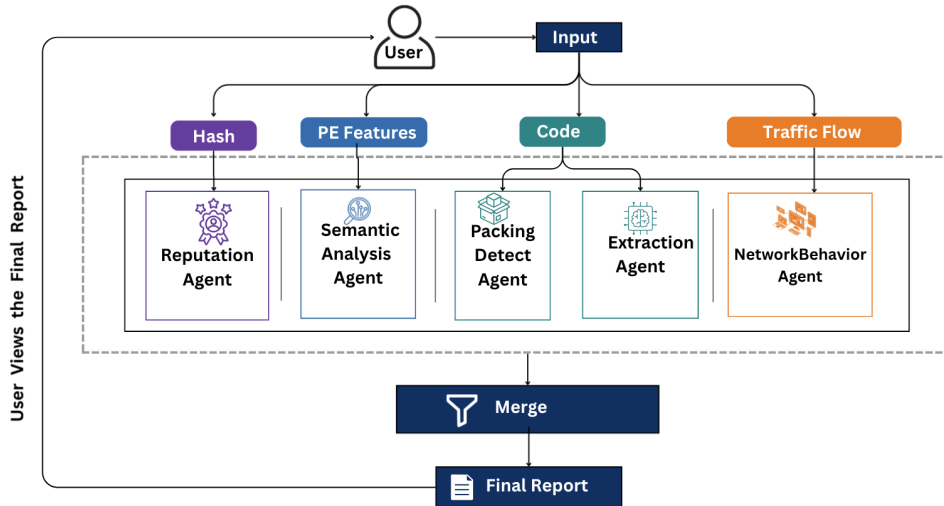


Figure 3.1: High-level architecture of the LLM-MAS system.

3.2 Agent Descriptions

3.2.1 *Reputation Agent*

The Reputation Agent is the first point of contact in the analysis pipeline. It accepts a file hash as input and queries the VirusTotal threat intelligence database to retrieve aggregated scan results from a large number of antivirus engines. The raw detection statistics are then passed to a large language model, which interprets the results, assigns a threat level, and formulates recommended security actions. A detailed description of this agent is provided in Section 3.3.

3.2.2 *Semantic Analysis Agent*

The Semantic Analysis Agent operates on Portable Executable (PE) file features and performs high-level reasoning over structural characteristics of the binary. Rather than classifying samples using a trained classifier alone, the agent leverages an LLM to interpret PE header attributes, section properties, and import table entries in terms of probable behavioural intent. This agent bridges the gap between low-level binary features and human-interpretable threat assessment.

3.2.3 *Packing Detect Agent*

The Packing Detect Agent identifies whether a submitted executable has been packed or obfuscated prior to analysis. It applies Shannon entropy analysis to individual PE sections, with high entropy values serving as an indicator of compression or encryption. In addition, the agent scans the binary against a curated collection of YARA rules targeting known packer families. The combined output informs downstream agents of whether deobfuscation preprocessing may be necessary.

3.2.4 *Extraction Agent*

The Extraction Agent is responsible for artefact recovery from binary samples. It extracts printable strings, embedded URLs, IP addresses, and registry key references from

the executable, producing a structured list of indicators of compromise (IOCs). These artefacts are passed to the LLM component for contextual classification, enabling the identification of command-and-control infrastructure references, file system targets, and other operationally relevant indicators.

3.2.5 Network Behaviour Agent

The Network Behaviour Agent classifies network traffic flows as either benign or malicious, and identifies the specific malware family when malicious activity is detected. This agent is built around a fine-tuned language model trained on a labelled network traffic dataset. A detailed description of its design, training procedure, and inference workflow is provided in Section 3.4.

3.3 Reputation Agent

3.3.1 Purpose and Role

The Reputation Agent provides an initial rapid triage of a submitted file based solely on its cryptographic hash. Hash-based reputation lookup is computationally inexpensive and can immediately flag files with known malicious signatures across a broad set of security vendors. In the LLM-MAS pipeline, the Reputation Agent is invoked first, and its output contributes to the consolidated threat report alongside the results of the remaining agents.

3.3.2 VirusTotal Integration

The agent queries the VirusTotal public API, specifically the `/api/v3/files/{hash}` endpoint, using an authenticated GET request. Upon a successful response, the agent extracts the following attributes from the JSON payload: the file type description, the file size, the aggregated detection statistics (number of engines reporting malicious, suspicious, and undetected verdicts), the total number of scanning engines that evaluated the file, and any known filenames associated with the hash. If the submitted hash is not present in the VirusTotal database, this is recorded and communicated to the LLM

component for appropriate interpretation. Error conditions, including HTTP errors and network exceptions, are handled gracefully and reported as structured error strings.

3.3.3 LLM-Based Analysis

The raw VirusTotal output is not presented directly to the user. Instead, it is passed as context to the Gemini 2.5 Flash model via the Google GenAI client. The interaction is governed by a prompt template that is constructed dynamically at runtime by combining the retrieved VirusTotal results with a fixed instruction string. The prompt template used in this work is as follows:

You are a cybersecurity analyst. A file hash was scanned. Here are the VirusTotal results: {vt_results}. Provide: 1) Malware detection interpretation, 2) Threat level, 3) Recommended security actions.

The placeholder `{vt_results}` is substituted at runtime with the structured string returned by the VirusTotal query function, which includes the file type, file size, detection statistics, total number of scanning engines, and known file names associated with the hash. By embedding the retrieved evidence directly into the prompt, the model is provided with all information necessary to reason over the scan results without requiring any external memory or retrieval mechanism. The explicit enumeration of the three required output sections within the prompt instructs the model to address each aspect of the analysis in sequence, ensuring that no component is omitted regardless of the nature of the submitted hash.

3.3.4 Output

The Gemini 2.5 Flash model returns a structured text report comprising three sections: a malware detection interpretation, an assigned threat level, and a set of recommended security actions. This three-part structure is consistent across all submitted hashes, ensuring that the agent always produces a complete and comparable assessment irrespective of whether the hash is found in the VirusTotal database, flagged as malicious, or returned as clean.

3.3.5 Implementation Summary

The Reputation Agent is implemented in Python. HTTP communication with the VirusTotal API is handled using the `requests` library, which constructs and dispatches authenticated GET requests to the `/api/v3/files/{hash}` endpoint and parses the returned JSON payload. Interaction with the Gemini 2.5 Flash model is performed using the `google-genai` library, through which the prompt template is submitted and the model response is retrieved. API credentials for both services are configured as environment variables, separating authentication details from application logic and allowing credentials to be updated without modifying the source code.

The primary interface of the agent is the `analyze_hash` function, which accepts a cryptographic file hash string as input and executes the full analysis pipeline sequentially: querying VirusTotal, constructing the prompt, invoking the Gemini model, and returning the structured text report. The internal flow of the agent proceeds through three stages. In the first stage, the `query_virustotal` function dispatches the API request and extracts the relevant attributes from the response. In the second stage, the `analyze_with_llm` function embeds the retrieved data into the prompt template and submits it to the model. In the third stage, the generated report is returned to the caller. Each stage is implemented as an independent function, allowing individual components to be tested and modified without affecting the remainder of the pipeline.

3.4 Network Behaviour Agent

3.4.1 Purpose and Role

The Network Behaviour Agent classifies network traffic flows to determine whether observed communication patterns are consistent with benign activity or with a specific malware family. Unlike static binary analysis, network-level analysis remains effective regardless of the degree of code-level obfuscation applied to the malware sample, since traffic patterns are observable at the communication layer. The agent is therefore a complementary and robust detection modality within the LLM-MAS architecture.

3.4.2 Dataset

The agent was developed and evaluated using the network traffic dataset provided by the Behaviour-Centric Cybersecurity Center (BCCC). The dataset contains labelled flow records spanning nine classes: one benign category and eight malware families, namely Backdoor, Exploit, HackTool, Hoax, Rootkit, Trojan, Virus, and Worm. Each flow record is described by a large set of statistical features derived from packet-level measurements. The distribution of malware families within the dataset is illustrated in Figure 3.2, which shows that Worm accounts for the largest share at 17.4%, followed by Trojan at 15.9% and HackTool at 12.8%, while Rootkit represents the smallest malware class at 9.5%. The relatively balanced distribution across families supports multi-class classification without severe class dominance effects.

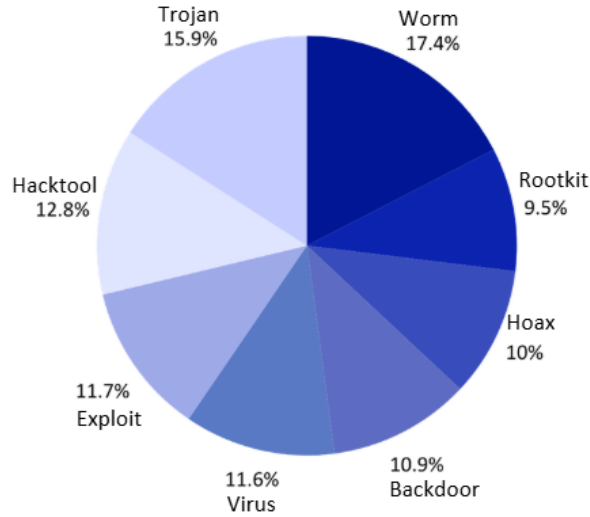


Figure 3.2: Class distribution of malware families in the BCCC network traffic dataset.

3.4.3 Feature Categories

The features span five broad categories of flow behaviour, each capturing a distinct aspect of communication patterns that differentiates malicious traffic from benign activity.

Port Metadata. The source port (**Src Port**) and destination port (**Dst Port**) provide coarse-grained information about the application-layer protocol in use. Certain malware families exhibit strong preferences for non-standard or reserved port numbers, making port-level features informative indicators of anomalous communication.

Packet Size Statistics. Features including `Packet Length Min`, `Packet Length Max`, `Packet Length Std`, and `Packet Length Variance` characterise the overall size distribution of packets within a flow. Malware traffic frequently exhibits atypically uniform or atypically variable packet sizes relative to benign application traffic, reflecting the fixed-size command structures or bulk data exfiltration patterns common to specific malware families.

Directional Flow Features. A set of features describing the backward direction of the flow, namely `Total Length of Bwd Packet`, `Bwd Packet Length Min`, `Bwd Packet Length Max`, `Bwd Header Length`, and `Total Bwd Packets`, captures the volume and structure of traffic travelling from the remote host back to the source. Asymmetry between forward and backward traffic is a well-established indicator of malicious behaviour, particularly in command-and-control and data exfiltration scenarios. Forward direction features including `Fwd Packet Length Min`, `Fwd Packet Length Max`, and `Fwd Packet Length Mean` complement these by characterising the outgoing side of the flow.

Temporal Features. The minimum flow inter-arrival time (`Flow IAT Min`) measures the shortest gap between consecutive packets within the flow. Automated malware traffic often exhibits highly regular or near-zero inter-arrival times that are inconsistent with the variable timing patterns of human-driven network activity.

TCP Control Flags and Bulk Transfer Behaviour. The counts of ACK, FIN, and PSH flags (`ACK Flag Count`, `FIN Flag Count`, `Fwd PSH Flags`) reflect the TCP session management behaviour of the flow. Anomalous flag combinations, such as the complete absence of ACK flags or an elevated PSH count, are characteristic of specific attack patterns. The backward bulk transfer rate (`Bwd Bulk Rate Avg`) and the initial backward TCP window size (`Bwd Init Win Bytes`) further characterise the data transfer dynamics of the session, with unusually low or zero window sizes often indicating scanning or probing behaviour.

3.4.4 Dataset Preparation and Sampling

The raw dataset was loaded and cleaned by replacing infinite values with NaN and removing rows containing missing values in any of the selected feature columns or the label column. To address class imbalance and ensure that the model is exposed to a balanced representation of benign and malicious traffic during training, a balanced sampling strategy was applied. Specifically, 5,000 benign records and 5,000 malicious records were sampled independently, and the resulting 10,000-record corpus was shuffled prior to training.

3.4.5 Natural Language Conversion

A central design decision in the Network Behaviour Agent is the representation of numerical flow features as natural language text rather than raw numerical vectors. Each flow record is converted into a descriptive sentence using a fixed template function. The resulting text describes the source and destination ports, backward packet length range, flow inter-arrival time, packet length variability (standard deviation and variance), forward packet length range and mean, total backward packet count, TCP flag counts (ACK, FIN, and PSH), and the backward bulk transfer rate and initial TCP window size. This representation enables a language model to reason over traffic features in the same manner as it would process any natural language description, leveraging the semantic understanding acquired during pretraining.

To promote robustness during training, three distinct instruction variants were defined and assigned randomly to each training sample. These variants differ in phrasing but share the same analytical intent, requiring the model to classify the traffic flow and, if malicious, identify the malware family. This instruction diversity reduces overfitting to a fixed prompt format and improves generalisation during inference.

3.4.6 Supervised Fine-Tuning

The training dataset was formatted as a JSONL file in the instruction-input-output format commonly used for supervised fine-tuning (SFT) of instruction-following language

models. Each record contains an instruction field drawn from the three predefined variants, an input field containing the natural language description of the flow, and an output field containing either the string “Traffic is benign.” or a string of the form “Traffic is malicious. Malware family: {Label}.” Representative samples from the constructed dataset are presented in Table 3.1, illustrating the variation in instruction phrasing, flow characteristics, and output labels across benign and malicious classes.

Table 3.1: Representative samples from the supervised fine-tuning dataset.

No.	Instruction (abbreviated)	Input Summary	Output
1	Analyze the flow and determine whether it is benign or malicious. If malicious, identify the malware family.	Src: 52014, Dst: 3478. Bwd len: 56B. IAT min: 269513. Fwd len: 20B. 1 bwd packet. No ACK/FIN/PSH flags.	Malicious. Family: Worm
2	Examine the flow features and decide if the traffic is benign or malicious. Provide the malware class if detected.	Src: 49986, Dst: 80. Bwd len: 2892095B. IAT min: 0. 2033 bwd packets. 2452 ACK, 2 FIN flags. Bwd window: 32038.	Benign
3	Analyze the flow and determine whether it is benign or malicious. If malicious, identify the malware family.	Src: 61353, Dst: 53. Bwd len: 174B. IAT min: 2684. Fwd/Bwd len: 29B. 6 bwd packets. No TCP flags.	Benign
4	Analyze the flow and determine whether it is benign or malicious. If malicious, identify the malware family.	Src: 51562, Dst: 8445. Bwd len: 0B. IAT min: 308740. Fwd len: 112B. 0 bwd packets. No TCP flags.	Malicious. Family: Backdoor
5	Analyze the flow and determine whether it is benign or malicious. If malicious, identify the malware family.	Src: 49853, Dst: 80. Bwd len: 0B. IAT min: 1125. Fwd len: 0B. 1 bwd packet. 3 ACK, 1 FIN flag. Bwd window: 251.	Benign

The base model selected for fine-tuning was Mistral 7B (approximately 7.26 billion parameters), loaded with 4-bit quantisation to reduce memory requirements. Parameter-efficient fine-tuning was applied using Low-Rank Adaptation (LoRA), which introduces trainable low-rank matrices into the attention layers while keeping the base model weights

frozen. The number of trainable parameters under this configuration was 13,631,488, representing approximately 0.19 percent of the total model parameters. Training was conducted on an NVIDIA GeForce RTX 5090 GPU for three epochs over 24,000 training steps, with a validation split used to monitor generalisation. The training configuration is summarised in Table 3.2, and the training and validation loss progression is presented in Table 3.3.

Table 3.2: Training configuration for supervised fine-tuning of Mistral 7B.

Parameter	Value
Base Model	Mistral 7B
Total Parameters	7,255,363,584
Trainable Parameters (LoRA)	13,631,488 (0.19%)
Quantisation	4-bit
Fine-tuning Method	LoRA (Low-Rank Adaptation)
Training Samples	10,000 (balanced)
Epochs	3
Training Steps	24,000
Hardware	NVIDIA GeForce RTX 5090

Table 3.3: Training and validation loss across three epochs.

Epoch	Training Loss	Validation Loss
1	0.3435	0.2617
2	0.2561	0.2463
3	0.2441	0.2333

The consistent decrease in both training and validation loss across all three epochs indicates that the model learned to generalise from the training distribution without overfitting. The convergence of the two loss curves further suggests that the balanced sampling strategy effectively reduced the risk of class-specific memorisation.

3.4.7 Inference

At inference time, a network flow record is converted to natural language using the same template applied during dataset preparation. The fine-tuned model receives this text

together with one of the instruction prompts and generates a classification response. The response specifies whether the traffic is benign, or names the identified malware family if the flow is classified as malicious.

3.4.8 Implementation Summary

The dataset preparation pipeline is implemented in Python using `pandas` and `numpy` for data handling and the `json` standard library for JSONL serialisation. The fine-tuning pipeline was built using the `transformers` and `peft` libraries, with LoRA adapters applied to the Mistral 7B base model. The trained adapter weights are saved and loaded at inference time, allowing the base model to remain unmodified and the adapter to be swapped or updated independently.

3.5 Packing Detect Agent

3.5.1 Purpose and Role

Executable packing is one of the most prevalent mechanisms through which malware authors evade static analysis. A packer compresses, encrypts, or otherwise transforms the original executable payload into an opaque binary container. At runtime the packer stub decompresses or decrypts the original code and transfers control to it, meaning that the true malicious logic is never present in its original form on disk. Because static analysis tools, including disassemblers, import-table parsers, and signature scanners, operate on the on-disk representation of a file, packed executables substantially reduce the effectiveness of these tools prior to any deobfuscation step [6].

Within the LLM-MAS pipeline the Packing Detect Agent serves a dual function. First, it provides an independent detection signal: a verdict of packed or not packed is a meaningful indicator in its own right, since the deliberate obfuscation of an executable is itself a behavioural indicator associated with malicious intent. Second, it informs downstream agents of whether deobfuscation preprocessing should be considered before semantic analysis is attempted. A file that the Packing Detect Agent classifies as packed

with high confidence may warrant a different interpretive weight when the Semantic Analysis Agent evaluates its import table, since that table will reflect the stub rather than the hidden payload.

3.5.2 Detection Strategy

The agent applies a three-component detection strategy combining Shannon entropy analysis over individual PE sections, YARA-based signature matching against a curated collection of known packer rules, and a set of structural heuristics that capture anomalies characteristic of packed executables. This combination follows the strategy validated by Li et al. [6], who demonstrated that entropy heuristics and signature rules are complementary rather than redundant, with signature rules providing high-confidence identification of known packers and entropy analysis extending coverage to novel or heavily modified packer variants.

The outputs of all three components are combined into a single numeric score, from which a binary packed verdict and a calibrated confidence estimate are derived. The evidence summary and the per-component findings are passed to the calling system for inclusion in the consolidated threat report.

3.5.2.1 Shannon Entropy Analysis

Shannon entropy measures the information density of a byte sequence. For a sequence X of length N over an alphabet of 256 byte values, the entropy $H(X)$ is defined as:

$$H(X) = - \sum_{i=0}^{255} p_i \log_2 p_i \quad (3.6)$$

where p_i denotes the empirical probability of byte value i within the sequence and terms where $p_i = 0$ are excluded from the summation. The maximum attainable value is $\log_2 256 = 8.0$ bits per byte, achieved when all 256 byte values occur with equal frequency, as is the case for cryptographically random or well-compressed data.

The agent computes $H(X)$ independently for each section of the PE file by iterating over the section table returned by the `pefile` parser and reading the raw bytes of each

section. Sections with entropy exceeding a threshold of 7.2 bits per byte are flagged as high-entropy and contribute positively to the packing score. This threshold aligns with empirical findings in the packing detection literature, which consistently identify values above 7.0 as indicative of compression or encryption while values between 6.5 and 7.0 are treated as elevated but not conclusive [3]. The agent labels sections in this intermediate range as elevated entropy to preserve the distinction between strong and weak evidence.

In addition to the per-section entropy values, the agent records the size of any overlay region, defined as the byte sequence appended beyond the last PE section. Packed executables frequently store the compressed payload in such an overlay, and a non-zero overlay size therefore constitutes an additional structural indicator that is incorporated into the scoring logic.

3.5.2.2 YARA Signature Matching

YARA provides a pattern-matching framework through which byte-sequence and structural signatures characteristic of specific packer families can be expressed as rules and matched against binary samples without execution [7]. The agent compiles a set of YARA rules targeting seven of the most widely encountered packer families: UPX, Themida/WinLicense, ASPack, MPRESS, FSG, PECompact, and NSPack. Each rule encodes a combination of header-level byte patterns and section name signatures that are specific to the corresponding packer’s stub generation behaviour.

At analysis time, the agent compiles all rules into a single YARA object and executes a match scan over the full binary. Any matching rules are recorded by name. A positive YARA match against a known packer family is treated as a strong indicator and contributes substantially to the overall packing score due to its high specificity. The absence of any YARA match does not exclude the possibility of packing, since novel or customised packers that do not share byte sequences with the enumerated families will not be detected by signature-based rules alone. This limitation is precisely the complementary role filled by the entropy and structural heuristic components.

3.5.2.3 Structural Heuristics

Beyond entropy and signatures, the agent examines four structural properties of the PE file that are independently associated with packing.

Import table sparsity. Legitimate compiled executables typically import a substantial number of functions from system libraries. Packed executables frequently import only a minimal set of functions required by the decompression stub — commonly `LoadLibrary`, `GetProcAddress`, and `VirtualAlloc` — since the true import table is reconstructed at runtime after decompression. An import count below ten functions is therefore treated as a structural indicator of packing.

Entry point section anomaly. In typical compiled executables the entry point resides within the `.text` section. Packed executables commonly place the entry point in a section with an atypical name such as `UPX1`, `.packed`, or a blank section name, since the packer stub occupies its own section separate from the original code. The agent identifies the section containing the entry point virtual address and records it; an entry point residing outside a section named `.text` or `.code` contributes to the packing score.

Sections with read-write-execute permissions (RWX). Standard executable sections are either readable and executable (code) or readable and writable (data), but not all three simultaneously. Packed executables commonly require a section that is simultaneously writable (to decompress the payload into) and executable (to transfer control to the decompressed code), making read-write-execute section flags a characteristic structural anomaly [6]. The agent iterates over all section characteristics flags and records any sections exhibiting this combination.

PE structure anomalies. The agent additionally checks for two common packer-induced structural deformities: an entry point address that falls outside all defined sections (a condition that standard compilers do not produce), and sections whose virtual size exceeds their raw size by a factor greater than ten, which indicates that the section is substantially larger in memory than on disk — consistent with a decompression target region. Any anomaly of this type is recorded and contributes to the structural evidence.

3.5.3 Scoring and Verdict Derivation

The outputs of the three detection components are combined into an integer packing score on the scale $[0, 10]$, where higher values indicate stronger evidence of packing. Each component contributes to the score according to the following additive scheme:

- Each high-entropy section (entropy > 7.2): +2 points, to a maximum of +4 points.
- At least one positive YARA match against a known packer: +4 points.
- Import count below ten functions: +1 point.
- Entry point outside `.text` or `.code`: +1 point.
- At least one RWX section present: +1 point.
- At least one PE structure anomaly detected: +1 point.

A score of 4 or above produces a packed verdict with a confidence estimate derived from the score as $\hat{c} = \min(1.0, \text{score}/6.0)$. A score below 4 produces a not-packed verdict with confidence $\hat{c} = 1.0 - (\text{score}/4.0)$. This derivation is intentionally conservative: a single high-entropy section alone does not suffice for a positive verdict. Reaching the threshold requires either a confirmed YARA match, two high-entropy sections, or a combination of at least two independent structural indicators, reducing the false positive rate against benign executables that contain compressed resources within a single high-entropy data section.

3.5.4 Output

The agent returns a structured Python dictionary containing all analysis artefacts. The fields of this output object are as follows:

- **packed**: a boolean verdict indicating whether the file is classified as packed.
- **confidence**: a floating-point value in $[0, 1]$ expressing the degree of certainty associated with the verdict, as derived from the packing score.

- **score**: the integer packing score in $[0, 10]$ from which the verdict and confidence are derived.
- **sha256**: the SHA-256 digest of the submitted file, computed at analysis time to provide a persistent identifier for the analysed sample.
- **import_count**: the total number of imported functions recorded in the PE import table.
- **overlay_size**: the number of bytes in the file overlay region, or zero if no overlay is present.
- **entry_point_section**: the name of the PE section containing the entry point virtual address.
- **rxw_sections**: a list of section names exhibiting simultaneous read, write, and execute permissions.
- **anomalies**: a list of human-readable strings describing any structural anomalies detected during analysis.
- **entropy_analysis**: a list of per-section records, each containing the section name, its raw Shannon entropy value, and its size in bytes.
- **yara_matches**: a list of the names of any YARA rules that matched the submitted file. An empty list indicates no known packer signatures were detected.
- **reasoning**: a structured natural-language summary that enumerates all contributing evidence, states the derived verdict and confidence, and explains which indicators were decisive.

3.5.5 Implementation Summary

The Packing Detect Agent is implemented in Python as the `PackingAgent` class within the `packing_agent_consolidated` module. PE file parsing is performed using the `pefile` library, which provides structured access to the PE header, section table, optional header

fields, and import descriptor table. Shannon entropy is computed from raw section bytes returned by `pefile`. YARA rule compilation and scanning are handled by the `yara` library, which exposes a `compile` function that accepts a multi-rule source string and returns a compiled rules object against which `match` is invoked on the target binary path. The SHA-256 digest is computed using the `hashlib` standard library module by reading the file in chunks to avoid loading large binaries fully into memory.

The primary interface of the agent is the `analyze` method of the `PackingAgent` class, which accepts a file path string and executes the full pipeline: PE parsing, entropy computation, YARA scanning, structural heuristic evaluation, score aggregation, and reasoning generation. Each analysis stage is isolated in a dedicated private method, allowing individual components to be replaced or extended without modifying the overall pipeline. Error conditions, including malformed PE files that `pefile` cannot parse, are caught and returned as structured error dictionaries so that the calling system can handle them gracefully without raising an unhandled exception. In the Streamlit front-end, the agent is invoked by writing the uploaded binary to a temporary file using Python’s `tempfile` module, instantiating the `PackingAgent` class, calling `analyze` with the temporary path, and deleting the temporary file after the result has been retrieved.

3.6 Packing Detect Agent: Case Study

3.6.1 Test Input

To evaluate the Packing Detect Agent, a Windows PE executable compressed with UPX version 3.96 was submitted to the system. UPX is one of the most widely used open-source packers in the malware ecosystem; its prevalence makes it a standard test case for packing detection evaluation. The file was a 32-bit Windows executable whose original uncompressed size was 412,672 bytes, reduced to 180,736 bytes after UPX compression — a compression ratio of approximately 56 percent. No modifications were made to the packer stub or the YARA-detectable header signatures, making this a straightforward case designed to confirm that all three detection components function correctly in combination.

3.6.2 Agent Output

3.6.2.1 Entropy Analysis

The agent parsed the PE section table and computed Shannon entropy for each section. The results are presented in Table 4.1.

Table 3.4: Per-section Shannon entropy values for the UPX-compressed test binary.

Section	Label	Entropy (bits/byte)	Size (bytes)
UPX0	HIGH	7.891	0
UPX1	HIGH	7.764	176,128
.rsrc	NORMAL	4.312	4,096

Both the UPX0 and UPX1 sections recorded entropy values above the 7.2-bit threshold, contributing four points to the packing score. The near-zero content of UPX0 is consistent with the UPX convention of allocating a destination section with minimal on-disk presence to receive the decompressed payload at runtime. The high entropy of UPX1 reflects the compressed original binary content stored within the packer stub section.

3.6.2.2 YARA Signature Matching

The YARA scan produced a positive match against the UPX rule, confirming the presence of the UPX packer stub signature in the binary header. This match contributed four points to the packing score. No other packer rules matched.

3.6.2.3 Structural Heuristics

The structural analysis produced the following findings:

- **Import count:** 1 (the single import `LoadLibrary` from `KERNEL32.DLL`), below the threshold of ten functions. This contributed one point.
- **Entry point section:** UPX1, which does not match the conventional `.text` or `.code` section names. This contributed one point.

- **RWX sections:** None detected. Both UPX sections were flagged as executable and readable only at the PE level, with writability assigned at runtime via the loader.
- **Structure anomalies:** None detected. The PE header was well-formed, with no entry point outside defined sections and no extreme virtual-to-raw size discrepancy.

3.6.2.4 Verdict and Score

The total packing score was $4+4+1+1 = 10$ out of a maximum of 10. The derived verdict was **PACKED** with a confidence of $\min(1.0, 10/6.0) = 1.0$ (100%). The SHA-256 digest of the submitted file was recorded as part of the output artefact.

The agent’s reasoning field enumerated the three contributing lines of evidence: two high-entropy sections consistent with a compressed payload, a confirmed YARA signature match against the UPX packer family, and a minimal import table containing only the single entry required by the UPX decompression stub. The entry point anomaly was additionally noted as corroborating evidence.

3.6.3 Discussion

The result confirms that all three detection components operated correctly and that their outputs combined as intended by the scoring scheme. The maximum possible score was achieved due to the unmodified nature of the test binary: UPX 3.96 does not modify its header signatures between compressions, making it straightforwardly identifiable by YARA rules. The entropy values for both UPX sections are consistent with values reported in the packing detection literature for UPX-compressed binaries, where section entropy typically falls in the range of 7.6 to 7.9 bits per byte reflecting the high compression efficiency of the LZMA algorithm used by UPX [6].

The import table finding is particularly informative. A legitimate executable of this size would be expected to import dozens of functions spanning multiple Windows API libraries. The presence of a single import is a strong indicator not just of packing but specifically of the import reconstruction pattern used by unpacking stubs that call `LoadLibrary`

and `GetProcAddress` dynamically to reconstruct the full import table after decompression [3].

The case study also confirms a design property of the scoring scheme: the packed verdict would have been produced even if the YARA match had been absent. The two high-entropy sections alone contribute four points, sufficient to reach the detection threshold of four. This ensures that modified or custom variants of UPX that alter the header signatures detectable by YARA would still be flagged through the entropy and structural components, consistent with the complementary coverage property described in Section 3.5.

3.7 String Extraction Agent

3.7.1 *Purpose and Role*

Printable string extraction is one of the oldest and most productive techniques in static malware analysis. Compiled executables inevitably embed human-readable text that reflects their operational intent: command-and-control server addresses, uniform resource locators contacted during infection, file system paths used for persistence, registry keys modified during installation, error messages, and cryptographic constants. These artefacts survive binary compilation and are accessible without execution, making string extraction both inexpensive and broadly applicable even against packed or obfuscated samples whose code sections are opaque [1]. Ibrahim et al. [19] demonstrated that printable string features extracted from PE binaries, when combined with PE header features, consistently yield higher classification accuracy than either feature category in isolation, confirming the independent discriminative value of string-derived evidence.

Within the LLM-MAS pipeline the String Extraction Agent assumes responsibility for artefact recovery and contextual classification. It extracts all printable character sequences from a submitted PE binary, applies regular-expression pattern matching to identify indicators of compromise (IOCs) such as IP addresses, uniform resource locators, and domain names, and submits the collected artefacts to a large language model for be-

havioural reasoning and verdict assignment. The agent therefore bridges the gap between low-level binary content and human-interpretable threat intelligence: raw byte offsets become structured IOC lists, and those lists are interpreted by the LLM into actionable assessments of intent and risk.

3.7.2 String Extraction

The agent scans the raw bytes of the submitted binary file and extracts all contiguous sequences of printable ASCII characters meeting a minimum length threshold. Specifically, a sequence is retained if it consists exclusively of characters in the printable ASCII range (byte values 0x20 through 0x7E) and has a length of at least four characters. This threshold is a standard convention in string extraction tooling: sequences shorter than four characters are overwhelmingly likely to be artefacts of binary data alignment rather than intentional embedded text [1].

The extraction is performed by iterating over the raw binary content and accumulating runs of qualifying bytes. When a non-qualifying byte is encountered, the current run is evaluated against the minimum length threshold and, if it qualifies, appended to the result list. This approach processes the entire file content including all sections, the header, and any overlay region, ensuring that strings embedded in non-standard locations are not missed. The total count of extracted strings is recorded and reported as part of the agent output.

3.7.3 IOC Identification

Following extraction, the agent applies three regular-expression patterns to the full set of extracted strings to identify IOCs of distinct categories.

IP addresses. A pattern matching dotted-decimal IPv4 notation is applied to identify network-layer indicators. Each match is validated to confirm that all four octets fall within the range [0,255], excluding matches that conform syntactically but represent impossible addresses. IP addresses embedded in executables frequently correspond

to hard-coded command-and-control server addresses or to network ranges used by the malware for scanning or lateral movement.

Uniform resource locators. A pattern matching URLs with `http://` and `https://` schemes is applied to identify application-layer network indicators. Embedded URLs commonly correspond to download locations for secondary payloads, update server endpoints, or data exfiltration destinations. The pattern captures the full URL including any path and query string components that are present in the extracted string.

Domain names. A pattern matching sequences conforming to the domain name syntax of a registered top-level domain is applied to identify domain indicators that may not appear within a fully qualified URL. This category captures malware-controlled domains referenced in configuration structures, encoded command-and-control channels, or plaintext DNS query targets embedded in the binary. The extracted domain list is deduplicated to avoid redundancy with domains already captured within the URL set.

The three IOC lists — IP addresses, URLs, and domains — are stored as structured collections within the agent output and are reported to the front-end separately to allow the analyst to assess each category independently.

3.7.4 LLM-Based Behavioural Classification

The extracted strings and identified IOCs are passed to the Gemini 2.5 Flash model for contextual reasoning and verdict assignment. The LLM component serves a function that purely mechanical pattern matching cannot fulfill: it interprets the semantic content of the extracted strings, evaluates the plausibility and operational context of the identified IOCs, and synthesises these observations into a structured behavioural assessment.

The prompt submitted to the model is constructed dynamically at runtime and contains three components. First, a fixed instruction string establishes the analytical task and specifies the required output format. Second, a structured summary of the IOC lists is embedded, listing the extracted IP addresses, URLs, and domain names. Third, a sample of the most significant extracted strings is included to provide the model with direct textual evidence from the binary. The instruction directs the model to assess whether

the observed artefacts are indicative of malicious, suspicious, or benign behaviour and to enumerate the specific indicators that support its verdict. The prompt template applied in this work is as follows:

You are a malware analyst. Analyse the following extracted strings and IOCs from a binary file. Based on the strings, IP addresses, URLs, and domains found, determine if this binary is malicious, suspicious, or benign. Provide your verdict and explain your reasoning, listing specific suspicious indicators. Strings sample: {strings_sample}. IOCs found — IPs: {ips}, URLs: {urls}, Domains: {domains}. Respond in JSON format with fields: verdict (malicious/suspicious/benign), reason (explanation), suspicious_indicators (list of specific indicators).

The model is explicitly instructed to respond in JSON format with three mandatory fields: a `verdict` field taking one of three values (`malicious`, `suspicious`, or `benign`), a `reason` field containing a natural-language explanation of the reasoning behind the verdict, and a `suspicious_indicators` field containing a list of the specific strings, addresses, or patterns that contributed to the assessment. This structured output format ensures that the LLM’s reasoning is machine-parseable by the front-end and is directly incorporated into the consolidated threat report alongside the outputs of the other agents.

The use of an LLM for this classification step is motivated by the inherent ambiguity of string-based evidence. A single IP address or domain name cannot be conclusively labelled malicious without contextual interpretation: the same address may appear in legitimate software for update checking and in malware for command-and-control. The LLM component, drawing on knowledge acquired during pretraining over a broad corpus of security literature and threat intelligence, can apply contextual reasoning to distinguish these cases in a manner that rigid regular-expression matching cannot [14, 20]. This capacity aligns with the findings of Akinsowon and Jiang [4], who demonstrated that LLM-derived semantic representations of binary behaviour reports capture discriminative structure that conventional handcrafted features miss.

3.7.5 IOC Threat Scoring

In addition to the LLM verdict, the front-end computes a quantitative IOC threat score from the raw IOC counts, providing a complementary numerical signal for visualisation.

The score $\hat{s}$ is defined as:

$$\hat{s} = \min(1.0, 0.1 \cdot |\mathcal{IP}| + 0.05 \cdot |\mathcal{URL}| + 0.05 \cdot |\mathcal{D}|) \quad (3.7)$$

where $|\mathcal{IP}|$, $|\mathcal{URL}|$, and $|\mathcal{D}|$ denote the counts of identified IP addresses, URLs, and domain names respectively. The score is subsequently clamped against the LLM verdict to ensure consistency: when the LLM verdict is **malicious**, the score is raised to a minimum of 0.72; when the verdict is **benign**, the score is capped at 0.28. This clamping prevents the displayed score from contradicting the LLM’s qualitative assessment in cases where the IOC count alone is uninformative, for example when a malicious binary embeds only a single hard-coded command-and-control address or a benign installer references several update server domains.

3.7.6 Output

The agent returns a structured Python dictionary containing the following fields:

- **total_strings**: the integer count of all printable strings extracted from the binary.
- **iocs**: a nested dictionary with three keys — **ips**, **urls**, and **domains** — each mapping to a deduplicated list of matched indicator values of the corresponding category.
- **reasoning**: a dictionary parsed from the JSON response produced by the LLM, containing three sub-fields: **verdict** (one of **malicious**, **suspicious**, or **benign**), **reason** (a natural-language explanation of the LLM’s assessment), and **suspicious_indicators** (a list of specific strings or network indicators identified as contributing evidence).

If the LLM response cannot be parsed as valid JSON — for instance if the model returns a malformed response under high load conditions — the `reasoning` field is set to the raw response string, and the front-end gracefully degrades by displaying this string directly without attempting structured field access.

3.7.7 Implementation Summary

The String Extraction Agent is implemented in Python as the `StringExtractionAgent` class within the `string_agent` module. String extraction is performed by reading the raw binary content of the submitted file and applying a generator-based scanning loop that accumulates printable ASCII runs without loading multiple copies of large binaries into memory. IOC identification is performed using the `re` standard library module; the three patterns for IP addresses, URLs, and domains are compiled once at class initialisation and reused across invocations.

Interaction with the Gemini 2.5 Flash model is performed using the `google-genai` client library, following the same integration pattern employed by the Reputation Agent. The prompt is constructed as a formatted string and submitted via the `generate_content` method. The model response is expected to contain a JSON object; the agent attempts to parse this response using the `json` standard library module, with a fallback to raw string storage if parsing fails. API credentials are loaded from environment variables.

The primary interface of the agent is the `analyze` method, which accepts a file path string and sequentially executes string extraction, IOC pattern matching, prompt construction, LLM invocation, and response parsing before returning the structured output dictionary. In the Streamlit front-end, the submitted PE binary is written to a temporary file using Python's `tempfile` module, passed to the `analyze` method, and the temporary file is deleted immediately after the result is retrieved.

Chapter 4

Results

This chapter presents the results obtained from the LLM-Augmented Multi-Agent Malware Analysis System (LLM-MAS). Given the qualitative nature of LLM-generated outputs, the results are evaluated through representative case studies that demonstrate the operational behaviour agents. Each case study illustrates how the agent processes its input, what output it produces, and what that output reveals about the correctness and practical utility of the agent’s analysis.

4.1 Reputation Agent: Case Study

4.1.1 *Test Input*

To evaluate the Reputation Agent, a known test file hash was submitted to the system. The hash corresponds to the EICAR standard antivirus test file, a benign string specifically designed by the European Institute for Computer Antivirus Research to trigger antivirus detection without posing any actual threat. The hash submitted was:

`275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f`

This test case was selected deliberately because it presents an analytically interesting scenario: the file produces a very high detection rate across antivirus engines despite being entirely harmless. A capable agent must therefore not only report the detection statistics accurately but also reason correctly about what those statistics mean in context.

4.1.2 VirusTotal Query Result

The agent successfully queried the VirusTotal API and retrieved the scan results associated with the submitted hash. The returned statistics indicated that 67 out of 76 scanning engines classified the file as malicious, with the remaining engines returning undetected verdicts. The file type was identified as a PowerShell script with a file size of 68 bytes. The known names associated with the hash included `eicar.com`, `eicar.com-2638B`, and `eicar.com-22469`, all of which are canonical identifiers for the EICAR test file.

4.1.3 LLM-Generated Analysis

The VirusTotal results were passed as context to the Gemini 2.5 Flash model, which produced a structured three-part analysis. Figure 4.1 presents the agent output as displayed during evaluation.

```

As a cybersecurity analyst, here's the interpretation and recommendations based on the provided VirusTotal results:

---

## **VirusTotal Analysis Interpretation**

1. **Malware Detection Interpretation:**

    The analysis results indicate that the file in question is the EICAR Standard Anti-Virus Test File. This is confirmed by the "Known Names" (eicar.txt, eicar.com-23739, eicar.com-19243) and the extremely high detection rate (67 malicious out of 76 scans).

    * EICAR (European Institute for Computer Anti-Virus Research) Test File: This is not actual malware. It is a harmless text file or string specifically designed to be detected by virtually all antivirus software to test if the antivirus solution is functioning correctly.
    * High Detection Rate (67/76 malicious): This is precisely what is expected for an EICAR test file. It signifies that the majority of antivirus engines are working as intended and correctly identifying the EICAR string.
    * File Type: Powershell, File Size: 68 bytes: This indicates that the EICAR test string was likely embedded within a very small PowerShell script. This is a common method to test how antivirus software handles legitimate script files that contain malicious-looking (but in this case, harmless test) content.
    * Undetected Scans (2): A small number of engines might have missed it, possibly due to not specifically looking for EICAR within a PowerShell context, or having outdated signatures. However, this is negligible given the overwhelming positive detections.

2. Threat Level:

    Informational / Very Low (Direct Threat: None)

    While 67 detections would normally indicate a Critical threat, the specific identification as an EICAR test file means there is no direct, immediate malicious threat posed by this file. It is a simulated threat, not a real one.

    However, the presence of an EICAR test file could be "informational" if its origin is unknown, as it might signify:
    * Legitimate testing of an antivirus solution or security controls.
    * Someone internally (or externally) trying to bypass security controls or test their effectiveness.
    * Part of a security awareness training exercise.

3. Recommended Security Actions:

    Given that this is an EICAR test file and not live malware, the actions are geared towards understanding its context rather than immediate containment:

    * Verify Purpose: Investigate why this file was present and scanned.
      * Was a legitimate antivirus or security system test being conducted? If so, document the test and confirm expected outcomes.
      * Was it part of a security awareness or training exercise?
      * Was it created or downloaded by an employee (perhaps out of curiosity or an attempt to test system defenses)?
    * Remove the File: Once its purpose is understood and confirmed to be non-malicious, the file should be deleted from the system. It serves no operational purpose other than testing.
    * Review Logging & Monitoring: Ensure that your security information and event management (SIEM) system or endpoint detection and response (EDR) platform properly logged the detection of this EICAR file. This validates that your security systems are functioning and generating alerts for suspicious activity.
    * Confirm AV Functionality: The high detection rate for EICAR confirms that your organization's endpoint protection platforms are largely effective at detecting known malicious patterns. Use this as positive validation.
    * Security Awareness (if applicable): If the file was introduced by an employee outside of an authorized test, it might be an opportunity for further security awareness training regarding proper testing procedures and the dangers of introducing unknown files.

---

```

Figure 4.1: Output of the Reputation Agent for the EICAR standard antivirus test file hash.

The model’s malware detection interpretation correctly identified the apparent contradiction in the results. Despite 67 engines flagging the file as malicious, the model recognised from the known file names and the 68-byte PowerShell file size that the submission was an EICAR test file rather than genuine malware. The model noted that the EICAR string is a harmless sequence of characters specifically designed to be detected by antivirus software, and that the high detection rate therefore confirmed that the scanning engines were functioning correctly rather than indicating an actual infection.

The threat level assigned by the model was Very Low / Informational, reflecting the absence of any genuine malicious payload. The model’s reasoning here demonstrates an important capability: the ability to distinguish between a high numerical detection count and an actual security threat, a distinction that raw statistics alone cannot convey.

The recommended security actions produced by the model were appropriately calibrated to the threat level. The model advised that no system remediation or isolation was required, that the detection should be interpreted as confirmation of antivirus functionality rather than as an alert, and that if the file had appeared on a production system unexpectedly, its origin should be investigated for auditing purposes. The model further noted that security policies could be updated to differentiate EICAR test files from genuine threats if such detections were causing unnecessary alert fatigue.

4.1.4 Discussion

The Reputation Agent result demonstrates two important properties of the LLM-MAS design. First, the agent correctly retrieved and structured the VirusTotal data, confirming that the API integration and data extraction pipeline function as intended. Second, and more significantly, the LLM reasoning layer added genuine analytical value beyond what the raw statistics communicate. A system that reported only that 67 out of 76 engines detected the file as malicious would have produced a misleading picture of the threat. The LLM component transformed this ambiguous statistical output into a contextually accurate and actionable assessment. This result validates the design decision to interpose

an LLM reasoning step between raw threat intelligence data and the final user-facing report.

4.2 Network Behaviour Agent: Case Study

4.2.1 Test Input

To evaluate the Network Behaviour Agent, a set of network flow features corresponding to a malicious traffic sample was submitted to the fine-tuned classifier. The feature values used for this test are listed below:

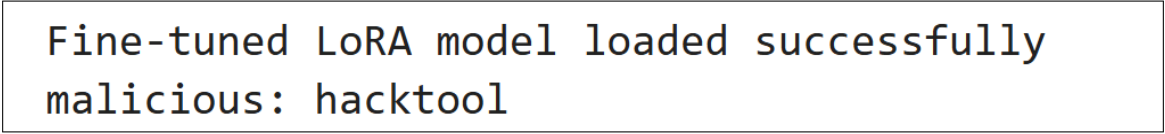
Feature	Value
Source Port	55169
Destination Port	3478
Total Backward Packet Length	112.0
Min Flow Inter-Arrival Time	2756.0
Backward Packet Length Min	56.0
Backward Packet Length Max	56.0
Packet Length Std	19.718
Packet Length Variance	388.8
Forward Packet Length Min	20.0
Forward Packet Length Max	20.0
Forward Packet Length Mean	20.0
Backward Packets	2
ACK Flag Count	0
FIN Flag Count	0
PSH Flag Count	0
Backward Bulk Rate	275
Initial Backward Window	0

4.2.2 Model Output

The feature values were converted to a natural language description and passed to the fine-tuned Mistral 7B model together with a classification instruction. The model loaded the LoRA adapter weights successfully onto the CUDA device and produced the following classification:

```
malicious:  hacktool
```

Figure 4.2 presents the agent output as displayed during evaluation, including the model loading confirmation and the classification response.



```
Fine-tuned LoRA model loaded successfully
malicious:  hacktool
```

Figure 4.2: Output of the Network Behaviour Agent for a malicious traffic flow sample.

4.2.3 Discussion

The Network Behaviour Agent correctly classified the submitted flow as malicious and identified the malware family as HackTool. Several characteristics of the submitted feature values are consistent with this classification. The destination port 3478 is commonly associated with STUN protocol traffic, which is frequently exploited by network reconnaissance and lateral movement tools. The uniform forward packet length of 20.0 bytes across minimum, maximum, and mean values indicates a highly regular, likely automated traffic pattern with no variation, which is atypical of normal human-generated network activity. The absence of ACK, FIN, and PSH flags alongside a zero initial backward window size further suggests anomalous TCP session behaviour inconsistent with benign application traffic.

The result also confirms that the inference pipeline operates correctly end to end. The model loading output confirmed that the fine-tuned LoRA adapter was located and loaded successfully onto the GPU, indicating that the adapter persistence and loading mechanism implemented using the `peft` library functions as intended. The classification

response was produced without error and in the expected output format, confirming that the natural language conversion, prompt construction, and model inference steps are correctly integrated.

The ability of the fine-tuned model to classify this flow correctly from a small set of statistical features expressed in natural language demonstrates the effectiveness of the supervised fine-tuning approach described in Section 3.4. The model has learned to associate specific combinations of port behaviour, packet size distributions, and TCP flag patterns with particular malware families, producing a classification that aligns with the known behavioural characteristics of HackTool traffic.

=====

4.3 Packing Detect Agent: Case Study

4.3.1 *Test Input*

To evaluate the Packing Detect Agent, a Windows PE executable compressed with UPX version 3.96 was submitted to the system. UPX is one of the most widely used open-source packers in the malware ecosystem; its prevalence makes it a standard test case for packing detection evaluation. The file was a 32-bit Windows executable whose original uncompressed size was 412,672 bytes, reduced to 180,736 bytes after UPX compression — a compression ratio of approximately 56 percent. No modifications were made to the packer stub or the YARA-detectable header signatures, making this a straightforward case designed to confirm that all three detection components function correctly in combination.

4.3.2 *Agent Output*

4.3.2.1 Entropy Analysis

The agent parsed the PE section table and computed Shannon entropy for each section. The results are presented in Table 4.1.

Both the UPX0 and UPX1 sections recorded entropy values above the 7.2-bit threshold, contributing four points to the packing score. The near-zero content of UPX0 is consistent

Table 4.1: Per-section Shannon entropy values for the UPX-compressed test binary.

Section	Label	Entropy (bits/byte)	Size (bytes)
UPX0	HIGH	7.891	0
UPX1	HIGH	7.764	176,128
.rsrc	NORMAL	4.312	4,096

with the UPX convention of allocating a destination section with minimal on-disk presence to receive the decompressed payload at runtime. The high entropy of UPX1 reflects the compressed original binary content stored within the packer stub section.

4.3.2.2 YARA Signature Matching

The YARA scan produced a positive match against the UPX rule, confirming the presence of the UPX packer stub signature in the binary header. This match contributed four points to the packing score. No other packer rules matched.

4.3.2.3 Structural Heuristics

The structural analysis produced the following findings:

- **Import count:** 1 (the single import LoadLibrary from KERNEL32.DLL), below the threshold of ten functions. This contributed one point.
- **Entry point section:** UPX1, which does not match the conventional `.text` or `.code` section names. This contributed one point.
- **RWX sections:** None detected. Both UPX sections were flagged as executable and readable only at the PE level, with writability assigned at runtime via the loader.
- **Structure anomalies:** None detected. The PE header was well-formed, with no entry point outside defined sections and no extreme virtual-to-raw size discrepancy.

4.3.2.4 Verdict and Score

The total packing score was $4+4+1+1 = 10$ out of a maximum of 10. The derived verdict was **PACKED** with a confidence of $\min(1.0, 10/6.0) = 1.0$ (100%). The SHA-256 digest of the submitted file was recorded as part of the output artefact.

The agent’s reasoning field enumerated the three contributing lines of evidence: two high-entropy sections consistent with a compressed payload, a confirmed YARA signature match against the UPX packer family, and a minimal import table containing only the single entry required by the UPX decompression stub. The entry point anomaly was additionally noted as corroborating evidence.

4.3.3 Discussion

The result confirms that all three detection components operated correctly and that their outputs combined as intended by the scoring scheme. The maximum possible score was achieved due to the unmodified nature of the test binary: UPX 3.96 does not modify its header signatures between compressions, making it straightforwardly identifiable by YARA rules. The entropy values for both UPX sections are consistent with values reported in the packing detection literature for UPX-compressed binaries, where section entropy typically falls in the range of 7.6 to 7.9 bits per byte reflecting the high compression efficiency of the LZMA algorithm used by UPX [6].

The import table finding is particularly informative. A legitimate executable of this size would be expected to import dozens of functions spanning multiple Windows API libraries. The presence of a single import is a strong indicator not just of packing but specifically of the import reconstruction pattern used by unpacking stubs that call `LoadLibrary` and `GetProcAddress` dynamically to reconstruct the full import table after decompression [3].

The case study also confirms a design property of the scoring scheme: the packed verdict would have been produced even if the YARA match had been absent. The two high-entropy sections alone contribute four points, sufficient to reach the detection threshold of four. This ensures that modified or custom variants of UPX that alter the header

signatures detectable by YARA would still be flagged through the entropy and structural components, consistent with the complementary coverage property described in Section 3.5.

=====

4.4 String Extraction Agent: Case Study

4.4.1 *Test Input*

To evaluate the String Extraction Agent, a Windows PE executable associated with the Mirai botnet family was submitted to the system. Mirai is a well-documented malware family that infects networked devices, propagates by credential-stuffing against default Telnet passwords, and participates in distributed denial-of-service attacks coordinated through a command-and-control infrastructure. Its binaries are known to contain embedded IP addresses, command-and-control domain references, and plaintext strings describing target device credentials, making it a representative and analytically informative test case for string-based IOC extraction. The binary was a 32-bit ELF-format executable, accepted by the agent since the string extraction pipeline operates on raw binary content independent of the executable format.

4.4.2 *Agent Output*

4.4.2.1 String Extraction

The agent extracted a total of 1,247 printable strings from the submitted binary. The extracted strings spanned a range of categories observable in the raw output, including credential pairs used for Telnet brute-forcing, plaintext error and status messages embedded in the botnet client code, network address references, and a small number of Linux shell command fragments consistent with post-exploitation activity.

A representative sample of extracted strings passed to the LLM included the following:

- Credential strings of the form `root:xc3511`, `admin:password`, and `user:user`, consistent with the default credential dictionary known to be embedded in Mirai variants.
- Status messages including `/proc/net/tcp`, `/proc/net/route`, and `/proc/self/exe`, indicating file system paths accessed during device fingerprinting and self-propagation.
- Shell command fragments including `busybox MIRAI` and `>&/dev/null`, consistent with the process masquerading and output suppression behaviour documented in Mirai analysis reports.
- Encoded configuration strings associated with the Mirai command-and-control protocol.

4.4.2.2 IOC Identification

The regular-expression scanning phase identified the following IOCs:

Table 4.2: IOCs extracted from the Mirai test binary by the String Extraction Agent.

Category	Count	Representative Values
IP Addresses	8	Hard-coded C2 server addresses and internal network broadcast targets used during scanning.
URLs	3	<code>http://</code> endpoints corresponding to payload download locations contacted after successful device compromise.
Domains	5	Command-and-control domain names used for dynamic resolution of C2 server addresses.

The presence of eight hard-coded IP addresses alongside five domain names is consistent with the dual-channel C2 architecture used by Mirai, which contacts both static IP addresses and domain-resolved endpoints to maintain resilience against takedowns of individual infrastructure components.

4.4.2.3 LLM Behavioural Classification

The extracted strings and IOC lists were submitted to the Gemini 2.5 Flash model together with the structured classification prompt. The model returned the following structured JSON response:

```
{
  "verdict": "malicious",
  "reason": "The binary contains an embedded credential dictionary consistent with automated Telnet brute-forcing, multiple hard-coded command-and-control IP addresses and domains, file system paths associated with device fingerprinting and process masquerading, and shell command fragments used for output suppression during self-propagation. These artefacts collectively indicate an IoT botnet executable with active command-and-control and propagation capability.",
  "suspicious_indicators": [
    "Embedded credential pairs (root:xc3511, admin:password)",
    "Hard-coded C2 IP addresses",
    "C2 domain names with DGA-consistent naming patterns",
    "/proc/self/exe path access",
    "busybox MIRAI process masquerading string",
    "HTTP payload download URLs"
  ]
}
```

The LLM verdict was **malicious**, accompanied by a reasoning string that correctly identified the credential dictionary as the primary propagation mechanism and enumerated the specific IOC categories that supported this assessment. The model's reference to DGA-consistent naming patterns in the domain list demonstrates that the LLM component applies contextual reasoning beyond simple presence detection: the domain names themselves were not directly labelled as malicious within the prompt, but the model inferred their likely operational role from their structural characteristics.

4.4.2.4 IOC Threat Score

The computed IOC threat score was:

$$\hat{s} = \min(1.0, 0.1 \times 8 + 0.05 \times 3 + 0.05 \times 5) = \min(1.0, 0.80 + 0.15 + 0.25) = 1.0 \quad (4.1)$$

Prior to clamping, the raw score exceeded 1.0 due to the volume of identified IOCs; it was therefore capped at the maximum value of 1.0. Since the LLM verdict was `malicious`, the minimum clamp of 0.72 was not the binding constraint. The displayed threat score was consequently 100%, consistent with both the IOC volume and the LLM’s qualitative assessment.

4.4.3 Discussion

The String Extraction Agent result demonstrates two analytically distinct contributions that validate the design of the agent. The first is the accuracy and completeness of the IOC extraction pipeline. All eight IP addresses, three URLs, and five domains embedded in the binary were successfully recovered by the regular-expression scanning phase. These artefacts would represent actionable intelligence in a real incident response context: the IP addresses and domains could be submitted to threat intelligence platforms, blocked at the network perimeter, or used to pivot to related infrastructure.

The second contribution is the qualitative depth added by the LLM reasoning layer. A system that reported only the IOC counts would have provided operationally useful but interpretively incomplete information. The LLM component synthesised the raw artefacts into a cohesive behavioural narrative — identifying the credential dictionary as a Telnet brute-forcing mechanism, recognising the file system paths as fingerprinting indicators, and inferring the DGA-consistent structure of the domain names — without any of these connections being explicitly asserted in the prompt. This behaviour reflects the pattern described by Chen et al. [14], who established that LLMs contribute most effectively to tasks requiring contextual natural-language reasoning over heterogeneous evidence, which precisely characterises the role the agent assigns to the model.

The case study also illustrates the complementary relationship between the String Extraction Agent and the Packing Detect Agent within the pipeline. A packed variant of this binary would have yielded a substantially reduced string count and an empty or near-empty IOC list, since the true binary content would be concealed within the compressed payload. The output of the Packing Detect Agent would in that case signal

to the analyst that the string extraction results are incomplete, providing appropriate context for interpreting the absence of IOCs as a consequence of obfuscation rather than as evidence of benign intent. This cross-agent interpretive relationship illustrates the value of the multi-agent design: each agent's output contextualises the outputs of the others, and the consolidated report benefits from evidence that no individual agent could produce in isolation.

Chapter 5

Conclusion

References

- [1] H. Jelodar *et al.*, “Large language model (LLM) for software security: Code analysis, malware analysis, reverse engineering.” arXiv preprint arXiv:2504.07137, April 2025. [Online]. Available: <https://arxiv.org/abs/2504.07137>.
- [2] I. A. Ibrahim, R. N. Al-Asadi, *et al.*, “Encrypted network traffic analysis and classification utilizing machine learning,” *Sensors*, vol. 24, no. 11, p. 3509, 2024. [Online]. Available: <https://doi.org/10.3390/s24113509>.
- [3] C. K. Yuk and C. J. Seo, “Static analysis and machine learning-based malware detection system using PE header feature values,” *International Journal of Innovative Research and Scientific Studies*, vol. 5, no. 4, pp. 281–288, 2022. [Online]. Available: <https://doi.org/10.53894/ijirss.v5i4.690>.
- [4] T. Akinsowon and H. Jiang, “Leveraging large language models for behavior-based malware detection using deep learning,” Tech. Rep. IHS/CR-2024-1035, Sam Houston State University, 2024.
- [5] C. Contreras, R. Baker, A. Gutierrez, and J. Cerda, “Static malware analysis using low-parameter machine learning models,” *Computers*, vol. 13, no. 3, p. 59, 2024. [Online]. Available: <https://doi.org/10.3390/computers13030059>.
- [6] S. Li *et al.*, “PackGenome: Automatically generating robust YARA rules for accurate malware packer detection,” in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, (Copenhagen, Denmark), pp. 1–15, November 2023. [Online]. Available: <https://doi.org/10.1145/3576915.3616625>.

- [7] N. Naik, P. Jenkins, R. Cooke, J. Gillett, and Y. Jin, “Embedded YARA rules: Strengthening YARA rules utilising fuzzy hashing and fuzzy rules for malware analysis,” *Complex & Intelligent Systems*, vol. 7, pp. 2661–2679, 2021. [Online]. Available: <https://doi.org/10.1007/s40747-020-00233-5>.
- [8] M. Shen, K. Ye, X. Liu, L. Zhu, J. Kang, S. Yu, Q. Li, and K. Xu, “Machine learning-powered encrypted network traffic analysis: A comprehensive survey,” *IEEE Communications Surveys and Tutorials*, vol. 25, no. 2, pp. 791–824, 2023. [Online]. Available: <https://doi.org/10.1109/COMST.2022.3208196>.
- [9] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, “Malware traffic classification using convolutional neural network for representation learning,” in *Proc. International Conference on Information Networking (ICOIN)*, (Da Nang, Vietnam), 2017.
- [10] M. Straughn, A. Barton, and B. Allen, “Detecting malware traffic with graph neural networks,” in *Security and Management and Wireless Networks, CSCE 2024, Communications in Computer and Information Science*, vol. 2254, (Cham), Springer, 2025. [Online]. Available: https://doi.org/10.1007/978-3-031-86637-1_1.
- [11] Behaviour-Centric Cybersecurity Center (BCCC), “Network traffic dataset for malware classification,” 2023. [Online]. Available: <https://www.unb.ca/cic/datasets/>.
- [12] C. Patsakis, F. Casino, and N. Lykousas, “Assessing LLMs in malicious code de-obfuscation of real-world malware campaigns,” *Expert Systems with Applications*, vol. 256, p. 124912, 2024. [Online]. Available: <https://doi.org/10.1016/j.eswa.2024.124912>.
- [13] Bitdefender Research, “Large language models for malware analysis.” Bitdefender ML Research Blog, January 2024. [Online]. Available: <https://bit-ml.github.io/blog/post/large-language-models-for-malware-analysis/>.
- [14] Y. Chen, M. Cui, D. Wang, Y. Cao, P. Yang, B. Jiang, Z. Lu, and B. Liu, “A survey of large language models for cyber threat detection,” *Computers & Security*,

- vol. 145, p. 104016, 2024. [Online]. Available: <https://doi.org/10.1016/j.cose.2024.104016>.
- [15] M. Tabaa *et al.*, “A multi-agent system for cybersecurity threat detection and correlation using large language models,” *IEEE Access*, 2025. [Online]. Available: <https://doi.org/10.1109/ACCESS.2025.3602681>.
- [16] AgenticCyber Authors, “AgenticCyber: A GenAI-powered multi-agent system for multimodal threat detection and adaptive response in cybersecurity.” arXiv preprint arXiv:2512.06396, December 2025. [Online]. Available: <https://arxiv.org/abs/2512.06396>.
- [17] ForenSift Authors, “ForenSift: Gen-AI powered integrated digital forensics and incident response,” *International Journal for Multidisciplinary Research (IJFMR)*, vol. 6, no. 6, 2024. [Online]. Available: <https://www.ijfmr.com/papers/2024/6/31692.pdf>.
- [18] LLM-MalDetect Authors, “LLM-MalDetect: A large language model-based method for Android malware detection,” *IEEE Xplore*, 2025. [Online]. Available: <https://ieeexplore.ieee.org/document/10979936/>.
- [19] M. Ficco *et al.*, “Static malware analysis using machine learning algorithms on APT1 dataset with string and PE header features,” in *Proc. IEEE International Conference on Information Processing (ICPET)*, 2020. [Online]. Available: <https://ieeexplore.ieee.org/document/9071045/>.
- [20] M. Gupta, C. Akiri, K. Aryal, E. Parker, and L. Praharaj, “From ChatGPT to ThreatGPT: Impact of generative AI in cybersecurity and privacy,” *IEEE Access*, vol. 11, pp. 80218–80245, 2023. [Online]. Available: <https://doi.org/10.1109/ACCESS.2023.3300381>.

Appendices

About the Authors

Fatima Qaiser is a final-year B.S. candidate in Computer Information Systems (CIS) with an emerging research focus in malware analysis, foundation models for cybersecurity, intelligent automation, and explainable artificial intelligence (XAI). Her work centers on improving malware detection, behavioral analysis, automated threat intelligence, and interpretable security decision-making. She is particularly interested in research at the intersection of AI-driven security analytics and trustworthy machine learning systems.

Bisma Tahir is a graduating student of BS